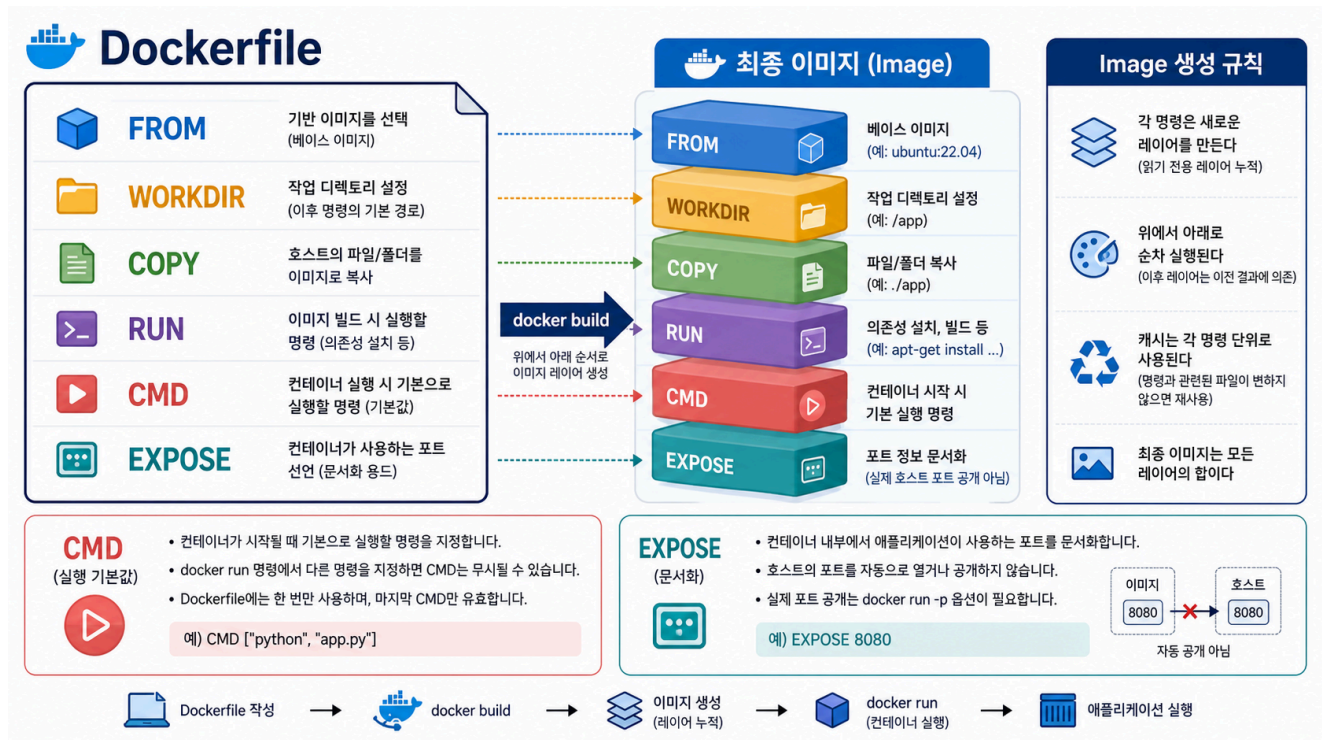


2교시: Dockerfile contract - FROM, WORKDIR, COPY, EXPOSE, CMD



수업 목표

- Dockerfile을 단순 문법 목록이 아니라 image를 만드는 계약서로 읽는다.
- FROM, WORKDIR, COPY, EXPOSE, CMD가 각각 어떤 실행 조건을 정하는지 설명한다.
- 제공된 week2/day3/labs/static-site/Dockerfile을 줄 단위로 해석한다.

왜 Dockerfile을 먼저 읽어야 하는가

Dockerfile은 docker build가 image를 만들 때 읽는 recipe다. 그런데 운영 관점에서는 단순 recipe를 넘어 실행 계약서 역할을 한다. 어떤 base image 위에서 실행되는지, 어떤 파일이 image 안에 들어가는지, container가 어떤 process로 시작되는지, 어떤 port를 쓸 것으로 기대하는지 Dockerfile에 드러난다.

Dockerfile을 읽지 않고 docker build만 실행하면, build가 성공해도 image가 무엇을 포함하고 어떻게 실행되는지 설명할 수 없다. 그래서 build 전에 Dockerfile을 먼저 읽는다.

실습 대상 Dockerfile

```
cd week2/day3/labs/static-site
sed -n '1,120p' Dockerfile
```

Expected:

```
FROM nginx:1.27-alpine
WORKDIR /usr/share/nginx/html
COPY index.html ./index.html
COPY styles.css ./styles.css
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

요소별 설명

1. FROM nginx:1.27-alpine

FROM은 image의 출발점을 정한다. 여기서는 nginx official image 중 1.27-alpine tag를 base로 사용한다. 즉, 우리는 웹서버를 직접 설치하지 않고 nginx가 이미 들어 있는 image 위에 HTML/CSS 파일만 얹는다.

운영 해석은 두 가지다. 첫째, 이 앱은 nginx runtime에 의존한다. 둘째, 1.27-alpine이라는 tag를 사용하므로 image size가 비교적 작고 Alpine 기반이라는 특성이 있다. 다만 tag는 content 자체가 아니므로 provenance나 재현성이 중요해지면 digest도 함께 봐야 한다.

2. WORKDIR /usr/share/nginx/html

WORKDIR은 이후 instruction의 기준 directory를 정한다. 이 Dockerfile에서는 nginx가 정적 파일을 제공하는 기본 위치인 /usr/share/nginx/html을 작업 directory로 둔다.

그래서 뒤의 COPY index.html ./index.html에서 ./index.html은 image 안의 /usr/share/nginx/html/index.html을 의미한다. 학생이 ./를 host 현재 directory로 오해하지 않도록 분명히 말해야 한다. COPY의 왼쪽은 build context 기준 host 파일이고, 오른쪽은 image 안의 target path다.

3. COPY index.html ./index.html

COPY는 build context 안의 파일을 image filesystem 안으로 복사한다. 이 줄은 lab directory에 있는 index.html을 image 안의 nginx html directory로 복사한다.

이 instruction이 실패하는 대표 이유는 index.html이 build context 안에 없기 때문이다. 예를 들어 다른 directory에서 build하거나, 파일명을 바꾸거나, static-site-broken에서 index.html을 지우면 COPY 단계에서 build가 실패한다. 이 실패는 runtime 문제가 아니라 build 단계 문제다.

4. COPY styles.css ./styles.css

이 줄은 CSS 파일을 image 안에 넣는다. CSS가 없어도 HTML만 있으면 HTTP 200은 나올 수 있다. 하지만 페이지가 기대한 모습으로 보이지 않는다. 따라서 verify는 HTTP status만 보는 것이 아니라 HTML 본문과 필요한 정적 파일 포함 여부까지 확인해야 한다.

COPY index.html과 COPY styles.css를 분리하면 어떤 파일 변경이 어떤 layer에 영향을 줬는지 보기 쉽다. 나중에 cache check에서 source 변경 후 어떤 COPY 단계가 다시 실행되는지 확인할 수 있다.

5. EXPOSE 80

EXPOSE는 container 내부에서 이 image가 80번 port를 사용할 것으로 기대한다고 문서화한다. 하지만 host의 port를 여는 명령이 아니다. 학생들이 가장 자주 헷갈리는 부분이다.

host에서 접근하려면 `docker run -p 18083:80 ...`처럼 port publish를 해야 한다. 여기서 18083은 host port이고 80은 container port다. EXPOSE 80이 있으니 run에서 container 쪽 port는 80을 사용해야 한다.

6. CMD ["nginx", "-g", "daemon off;"]

CMD는 container가 시작될 때 기본으로 실행할 command다. nginx는 기본적으로 daemon/background 방식으로 동작할 수 있는데, container에서는 main process가 살아 있어야 container도 살아 있다. 그래서 `daemon off;`로 foreground 실행을 유지한다.

만약 CMD가 잘못되면 image build는 성공해도 container가 바로 종료될 수 있다. 이때는 `docker ps -a`와 `docker logs`를 봐야 한다. 즉 CMD 문제는 build 문제가 아니라 run 단계 문제다.

확인 명령

base image의 기본 metadata를 확인한다.

```
docker image inspect nginx:1.27-alpine --format "{{json .Config.ExposedPorts}}
{{json .Config.Cmd}}"
```

Expected pattern:

```
{"80/tcp":{}} ["nginx" "-g" "daemon off;"]
```

제공된 lab image를 build한 뒤에는 history에서 Dockerfile instruction 흔적을 다시 확인한다.

```
docker build -t paperclip-static-site:day3 .
docker history paperclip-static-site:day3
```

Expected pattern:

```
COPY styles.css ./styles.css
COPY index.html ./index.html
WORKDIR /usr/share/nginx/html
EXPOSE map[80/tcp:{}]
CMD ["nginx" "-g" "daemon off;"]
```

학생이 말할 수 있어야 하는 문장

FROM은 base image를 정한다.
WORKDIR은 image 안에서 이후 명령의 기준 directory를 정한다.
COPY의 왼쪽은 build context 기준 source이고, 오른쪽은 image 안 target이다.
EXPOSE는 container port 문서화이며 host port publish가 아니다.
CMD는 container 시작 시 실행될 main process를 정한다.

핵심 포인트

Dockerfile은 build 전에 읽어야 한다. 각 instruction을 설명할 수 있어야 build 실패, run 실패, verify 실패를 분리할 수 있다.

다음 연결

다음 교시는 COPY가 의존하는 build context와 .dockerignore를 다룬다. Dockerfile을 읽었으니 이제 Dockerfile이 참조하는 입력 경계를 확인한다.